

REKOLT Planters' Cooperative Produce Tracker

Objective 4 — On-Paper Design

Programming II · ALCHE · design-v1 (paper design, pre-implementation)

1. Scope Statement

This document is the paper design produced before any Objective 5 code is written. It covers the desktop console application built for REKOLT Planters' Cooperative to replace the current manual weigh-slip and calculator process. The system records deliveries at the point of collection, applies a fixed payment rule set identically to every delivery, answers the season-level questions the treasurer is currently asked at the end of the season, and produces a single Word document containing one signed payment statement per member.

Out of scope: persistence of delivery data between program runs (data lives only in the in-memory collections for the duration of one execution); any network, multi-user, or concurrent-access behaviour; any currency other than MUR; any produce category beyond the four fixed codes.

1.1 Assumptions

- A single clerk or treasurer operates the console at a time; there is no concurrent multi-user access to consider.
- One program run corresponds to one season. Restarting the program starts a new, empty season.
- The four produce codes (MZE, BNS, POT, TEA) and their base prices are fixed for the season and do not change mid-run.
- The treasurer reviews the on-screen season figures before choosing to generate the final report, so the report generation step can safely assume the data entered is what should be reported.
- Only one season report is generated per run in the normal workflow, though the menu does not prevent generating it more than once (each generation appends a new run-log line, per the spec).

2. Functional Requirements

- FR1. The system shall record a delivery given a valid member identifier, member name, produce code, mass, quality score and week.
- FR2. The system shall validate every field individually and re-prompt with a short reason on invalid input, without terminating the program.
- FR3. The system shall compute the net payable for a delivery using the five-step rule: base value, grade multiplier, category multiplier, commission, transport levy.
- FR4. The system shall assign a Grade of A, B, C or REJECT from the quality score using the exact boundaries 85–100 / 70–84 / 50–69 / below 50.
- FR5. A delivery graded REJECT shall still be recorded and still count towards volume statistics, but its net payable shall be exactly zero with no commission or transport levy deducted.
- FR6. The system shall display, on request, the total payment per member for the season.
- FR7. The system shall display, on request, a weekly volume grid broken down by produce code.
- FR8. The system shall display, on request, the top five deliveries by net value.
- FR9. The system shall generate a single Word document with one section per member, each starting on a new page, containing that member's delivery table and net payable in bold with a signature line.
- FR10. The season report's closing totals shall reconcile exactly with the sum of the individual member net payables.
- FR11. Each report generation shall append one timestamped line to output/run-log.txt.
- FR12. The user shall be able to exit the application cleanly from the menu at any time.

3. Non-Functional Requirements

- NFR1. The application shall never terminate with an uncaught exception because of user input, regardless of what is typed.
- NFR2. Monetary values shall be rounded to two decimal places only at the point of display; all intermediate arithmetic shall retain full double precision.
- NFR3. The codebase shall be organised into app, model, service and util packages, with no package depending in the wrong direction (model shall not depend on app or service).
- NFR4. The polymorphic delivery-processing path shall contain no instance of checks and no downcasting.
- NFR5. The project shall build and run from a clean clone using a single documented command.
- NFR6. Exceptions raised while writing the season report shall be caught by specific type and reported with a message the user can act on, never a bare catch-all.

4. Noun–Verb Analysis

Nouns extracted from the brief were screened for distinct state and behaviour. Those kept as classes, and three genuine candidates rejected with reasons, are below.

4.1 Classes kept

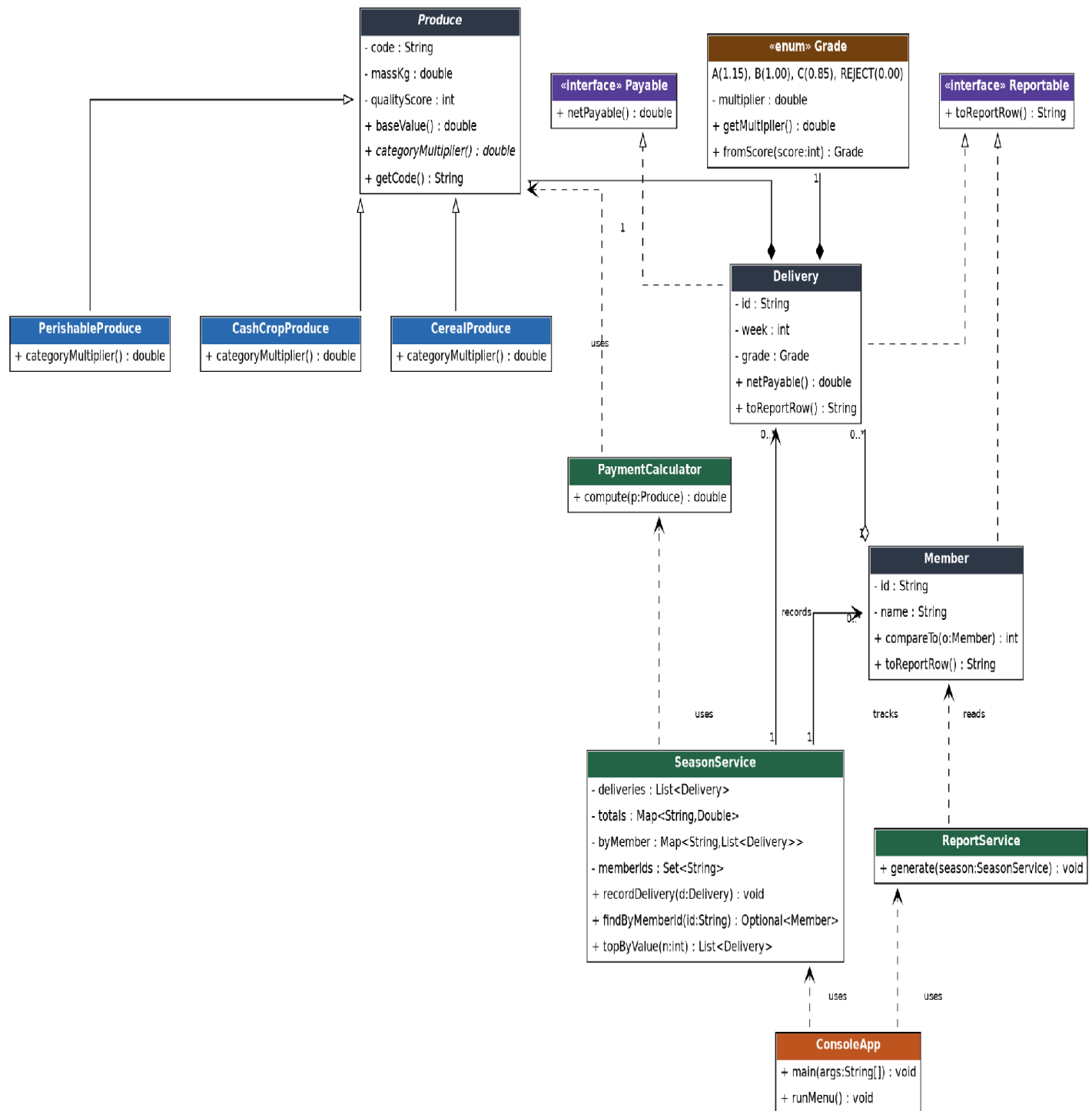
Class	Responsibility
Produce (abstract)	Shared valuation contract across crop categories
CerealProduce / PerishableProduce / CashCropProduce	Category-specific multiplier ($\times 1.00$ / $\times 0.90$ / $\times 1.10$)
Grade (enum)	Carries its own multiplier per letter grade, incl. REJECT
Member	Identity and natural ordering of a cooperative member
Deliverie	One weighed, graded load; computes its own net payable
Payable / Reportable (interfaces)	Capability contracts, not concrete types
PaymentCalculator	Stateless five-step arithmetic
SeasonService	Owens the season's collections and aggregation
ReportService	Builds the .docx from the season data
tracker	Menu loop and I/O only

4.2 Candidates rejected

Candidate noun	Reason rejected
Transaction	No behaviour or state distinct from Delivery — would be a redundant wrapper with no separate lifecycle.
Cooperative	There is exactly one, for one season; a class would add ceremony with no payoff. The season is represented by SeasonService's collections instead.
PriceList	The four prices are fixed reference data, not an entity with behaviour — a constant map inside PaymentCalculator serves the same purpose without an extra type.

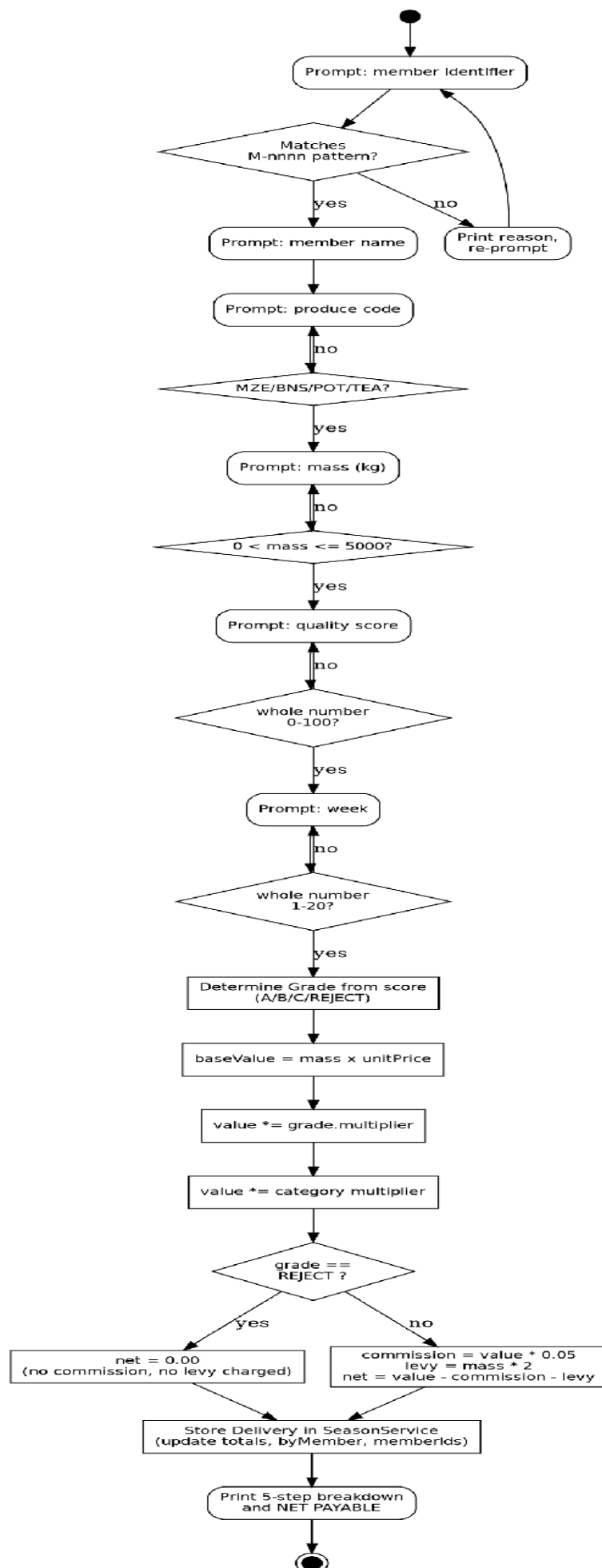
5. UML Class Diagram

Solid arrow + hollow triangle = inheritance. Dashed arrow + hollow triangle = interface realisation. Filled diamond = composition (the part cannot outlive the whole — a Delivery's Produce and Grade have no meaning detached from that Delivery). Hollow diamond = aggregation (a Member's Deliveries could conceptually be re-grouped without destroying them). Plain arrow = association/dependency, labelled with its verb.



6. Activity Diagram — Record a Delivery

This traces the full "Record a delivery" menu option end to end, including every validation loop and the REJECT branch of the payment calculation.



7. Pseudocode — Net Payment Calculation

The REJECT case is an explicit early return, not an incidental side-effect of a zero multiplier — this matters because the transport levy is calculated from mass, not from value, so a REJECT load would otherwise still be charged a levy despite the spec's "no deductions are taken from it" rule.

```
FUNCTION computeNetPayable(produce, massKg, qualityScore):
    unitPrice = priceOf(produce.code)           // fixed per crop, MUR/kg
    baseValue = massKg * unitPrice
    grade     = Grade.fromScore(qualityScore) // A / B / C / REJECT

    IF grade == REJECT THEN
        RETURN 0.00 // recorded, counted in volume stats,
                   // but zero value AND zero deductions
    END IF

    gradedValue = baseValue * grade.getMultiplier()
    categoryValue = gradedValue * produce.categoryMultiplier()
    commission = categoryValue * 0.05
    transportLevy = massKg * 2
    netPayable = categoryValue - commission - transportLevy

    RETURN netPayable // unrounded; round only when displayed
END FUNCTION
```

8. Data Dictionary

Field	Type	Unit	Range	Validation rule
Member.id	String	—	M-0000 – M-9999	Must match ^M-\d{4}\$; empty is rejected
Member.name	String	—	any non-empty text	Rejected if empty or whitespace-only
Delivery.id	String	—	system-generated	Not user input; unique per delivery
Delivery.week	int	week no.	1–20	Whole number in range; rejected otherwise
Delivery.massKg	double	kg	$0 < m \leq 5000$	Rejected if ≤ 0 , > 5000 , or not numeric
Delivery.qualityScore	int	score	0–100	Whole number in range; rejected otherwise
Delivery.grade	Grade (enum)	—	A, B, C, REJECT	Derived from qualityScore boundaries, not user input
Produce.code	String	—	MZE, BNS, POT, TEA	Case-insensitive on input, normalised to upper-case
Produce.unitPrice	double	MUR/kg	30 / 90 / 45 / 25	Fixed constant per code, not user input
Delivery.commission	double	MUR	= 5% of category value	Computed; zero when grade is REJECT
Delivery.transportLevy	double	MUR	= massKg $\times$ 2	Computed; zero when grade is REJECT
Delivery.netPayable	double	MUR	≥ 0	Computed; rounded to 2dp on display only

9. Design Rationale

9.1 Abstract Produce subclassed by category, not a category field with a switch

An abstract Produce with CerealProduce/PerishableProduce/CashCropProduce pushes the category multiplier into the type system: a crop without a category simply cannot be constructed, and the polymorphic List<Produce> pass in Objective 5 needs zero instanceof to dispatch correctly. The rejected alternative — one Produce class with a category field and a switch in the valuation method — was set aside because it recreates the exact if/else sprawl the abstraction step exists to remove, and a fifth crop later would mean editing a shared switch rather than adding one new class.

9.2 Two narrow interfaces (Payable, Reportable) rather than one combined interface

Delivery genuinely needs both capabilities; Member only needs to report, not to independently expose a payable amount outside its deliveries. Two narrow interfaces let each class implement only what it actually does. The rejected alternative — a single combined interface — was set aside because it would force any future Reportable-only class to carry a stub netPayable() it doesn't logically own.

9.3 Per-member deliveries held in SeasonService (Map<String, List<Delivery>>), not a list field on Member

Keeping the grouping in SeasonService means exactly one place is responsible for season-level bookkeeping, and Delivery creation doesn't need a back-reference into Member's internals. The rejected alternative — Member owning its own List<Delivery> directly — was set aside because every piece of code that creates a Delivery would then also need to reach into the right Member object to append it, coupling delivery creation to member state it shouldn't need to know about.